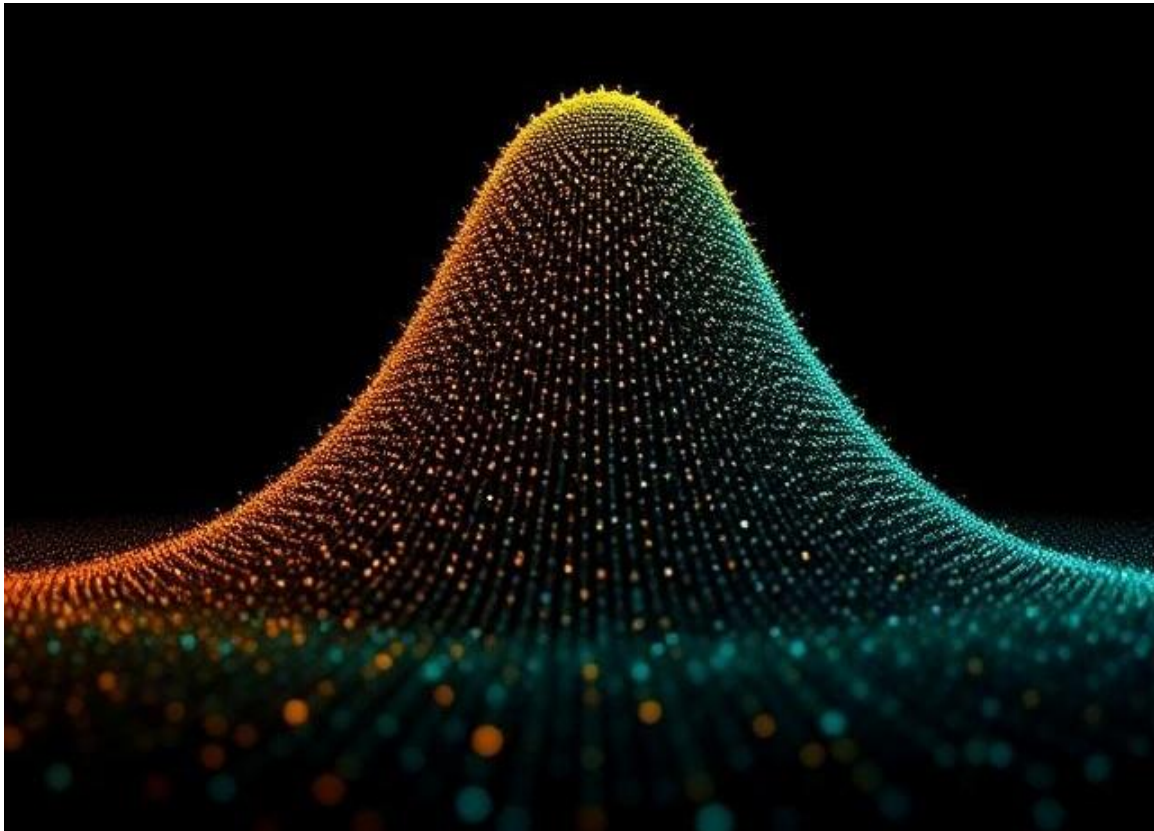


How We Built a Regulatory-Grade Financial Risk Platform End-to-End with Claude Code

385 risk functions, 649 commits, and a native MCP server—how an AI agent co-authored a production-ready quantitative finance ecosystem.



Somewhere in the first draft of pyvar's Solvency II module, one compounding factor went missing. It was a small bug — a few missing lines in one formula. The consequence wasn't small: an insurer running it would have been told it needed roughly 79% less regulatory capital than Solvency II actually requires.

No human caught it by reading the code. It was caught because the code was made to prove itself — cross-validated against QuantLib and published worked examples, run and re-run against every domain until the numbers either matched or didn't. This time, they didn't.

That's the moment this article is really about: not that an AI agent wrote financial software, but what happened when that software got checked.

79%

how much required regulatory capital pyvar's first Solvency II draft understated — caught by its own validation loop before it ever reached production

pyvar.com is an open-source (Apache-2.0) engine for exactly this kind of high-stakes number — 385 functions covering Value-at-Risk, Expected Shortfall, credit risk, derivatives Greeks, and regulatory capital across Basel, Solvency II, EMIR, and more. What makes it worth writing about isn't the domain coverage on its own. It's that the whole thing — math kernels, deployment pipeline, and the validation suite that caught the bug above — was built end-to-end by Claude Code.

At a glance

385

functions,
8 domains

649

commits · 131
co-authored by Claude

4

regulatory bugs
caught pre-launch

23.6%

caveats disclosed
inline, not hidden

Apache-2.0

fully open
source

If that's all you came for, that's the shape of it. Everything below is the receipts — every number in this article is checkable against [git log](#), [CHANGELOG.md](#), [portal/functions.json](#), or a live API call, including three corrections we made to this article itself when a claim didn't survive being checked:

A note on "verified facts only": every number, function count, and bug description in this article is checkable against this repository — [git log](#), [CHANGELOG.md](#), [portal/functions.json](#), or a real PyPI/API call. Where a claim was assumed rather than verified during drafting, we ran it against the actual code and corrected it before publishing. Three of those corrections are told as part of the story below, not edited out — because the honesty mechanism this article describes only means something if it also applies to the article itself.

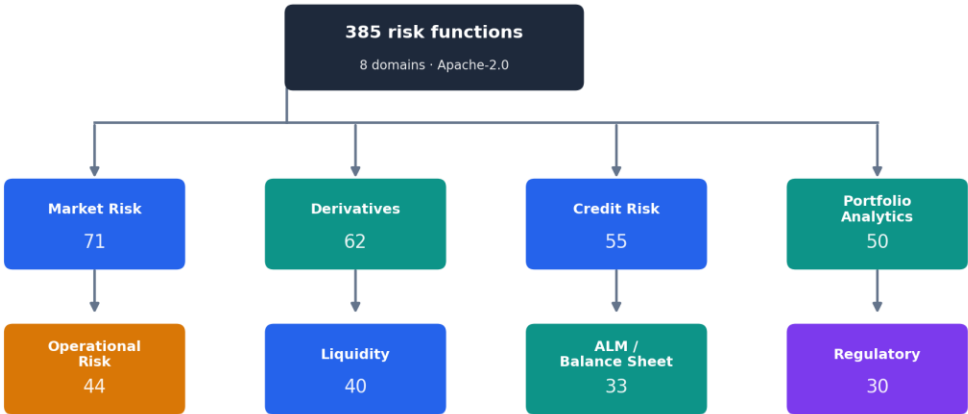
The problem this exists to solve

Regulatory-grade risk computation — Value-at-Risk, Expected Shortfall, Basel backtesting, FRTB capital, Solvency II SCR, derivatives Greeks — is normally locked inside proprietary vendor platforms: closed-source, expensive per seat, and

effectively unauditible by the risk teams whose regulatory capital depends on them. If a bank's SCR formula has a bug, nobody outside the vendor can see it, let alone fix it.

pyvar.com is the opposite bet: an Apache-2.0, open-source REST API exposing **385 risk functions across 8 domains**, JIT-accelerated with Numba, served through an async Celery/SQS pipeline on AWS. Every function's source is readable. Every regulatory constant is a line of code you can grep for.

What makes it a story worth telling isn't the domain coverage on its own — it's that the whole thing, from the first [CLAUDE.md](#) scaffold to the security review that preceded its public launch, was built with Claude Code. Not "Claude wrote some boilerplate." Claude Code wrote the Numba kernels, found real regulatory bugs before they shipped, built the CI/CD pipeline, and then built its own way back into the Claude ecosystem via an MCP server and a Jupyter extension.



Every regulatory constant is a line of code you can grep for.

Illustration 1 — how a request actually flows

This is the real path, not a simplified one. Three client surfaces (MCP, SDK, Jupyter) all converge on one [pyvar-client](#) SDK, so there's no duplicated auth/retry logic to keep in sync across them:



Only `var.compute` goes through the async job queue — it's the one function whose Monte Carlo path count makes submit-and-poll worth it. The other 384 are synchronous request/response, which is why `pyvar-jupyter`'s `%pyvar` magic can just return a result inline without a polling loop of its own — the SDK's `var.compute` already hides its own submit/poll cycle, and everything else was never async to begin with.

Bugs Claude Code found before anyone else could

The interesting part of "AI-built regulatory software" isn't that Claude Code wrote the code — it's what happened when that code got checked against real formulas. Four fixes from the `[0.1.0]` release notes, quoted exactly from `CHANGELOG.md`:

- **Solvency II SCR credit-risk formula (Art. 200–201)** — corrected an error that understated required capital by roughly 79%. Not a rounding error — a structurally wrong formula that would have told an insurer it needed less capital than Solvency II actually requires.
- **rBergomi kernel** — the model was missing the fractional-Brownian autocovariance structure that gives rough volatility models their name. Without it, "rBergomi" was just Bergomi.
- **EMIR clearing obligation scope** — financial counterparties were being evaluated per-asset-class instead of being in scope across all asset classes, which is what the regulation actually requires.

- **IRRBB standard shocks** — recalibrated to the BCBS d578 (2024) shock values; the prior implementation was running stale pre-2024 numbers.

None of these were caught by a human skimming the code. They were caught by building [tests/validation/](#) — a cross-validation suite checking pyvar's outputs against QuantLib and published worked examples — and then actually running it, repeatedly, against every domain, rather than trusting that "it compiled and returned a plausible-looking number" meant it was correct.

A fifth example, smaller in scope but from the same discipline, closer to this article's own publication date: PR #306 re-implemented 17 functions flagged with numerical caveats, and one of those re-implementations surfaced a real bug in the Monte Carlo CVaR optimizer's solver — again, found by re-deriving and re-running the numbers, not by inspection alone.

The caveat backlog, continued: 8 more fixes, 2 caught by reviewing its own work

PR #306 wasn't the end of the caveat backlog — it was the first pass. The follow-on work (PRs #314–#318) read every remaining caveat's full text across all 8 domains, sorted each into "an honest disclosure, nothing to fix" versus "a real, bounded gap against a named external standard," and fixed the eight that were the latter, spread across six of those domains:

- ``creditmetrics_portfolio_model`` went from a pass-through of the ordinary two-state Credit VaR engine to a genuine multi-state CreditMetrics model (Gupton, Finger & Bhatia 1997) — each obligor's simulated asset return now migrates through its own rating-transition row, not just default-or-survive.
- ``downturn_lgd_adjustment`` got the EBA/GL/2019/03 additive fallback formula wired in as an opt-in mode — the function's own docstring had documented that formula for a while; nobody had connected it to a code path.
- ``business_continuity_risk_score``'s `rpo_hours` parameter — accepted, validated, and doing precisely nothing to the score it was passed into — got a real RPO-breach severity to compare against.
- **CRR2 Art. 395(1)**'s EUR 150m institution-counterparty absolute alternative: `crr2_large_exposure_limit` accepted an `is_institution` flag whose own docstring said it "affects the absolute alternative limit," and then never applied it. Now it does.
- ``asset_swap_spread``'s `bond_price` parameter, previously accepted and silently unused, turns out to matter: the standard market convention (O'Kane, 2000, *Introduction to Asset Swaps*) prices the spread against a bond's actual dirty price, not against par — the two only agree when the bond happens to trade at par.
- ``bond_pricer_floating_rate`` gained a consistency check between `maturity` and the actual number of coupon periods being priced — previously the two could silently disagree.
- ``combined_stress_scenario`` gained an opt-in path shaped like BCBS 238's regulator-set retail deposit stability categories, instead of forcing every caller through one blended run-off rate a multi-category regulation can't be expressed through.
- ``transaction_cost_analysis`` gained Perold's (1988) missing opportunity-cost leg — the paper cost of the *unexecuted* portion of an order, the piece that was still absent even after an earlier pass had already added the delay-cost leg.

Every one of the eight is additive: pass nothing new, get the exact same output as before. That's not incidental — it's the same discipline PR #306 established, checked again here by exact dict-equality tests, not just "still runs" smoke tests.

The more interesting part is what a second pass — a code review, run before any of this merged — found wrong with the *first* pass:

Bug 1 — a parameter that looked used but wasn't. The new `creditmetrics_portfolio_model` docstring claimed `pd` "still drives the default threshold... even in multi-state mode." It didn't. The default threshold was computed entirely from the transition matrix; `pd` was validated for range and then never touched again. The included cross-check test didn't catch this because it happened to construct a transition matrix whose own default probability matched `pd` exactly — so the bug and the correct behaviour produced identical numbers in that one test, by coincidence. The fix makes `pd` genuinely override the

matrix's own default probability, via an affine rescale that preserves the matrix's migration shape while forcing the total default probability to match `pd` — and the regression test that proves it deliberately uses a transition matrix whose default probability is *wrong* (0.5, against a real `pd` of 1–8%), so the old bug would fail this test loudly if it ever came back.

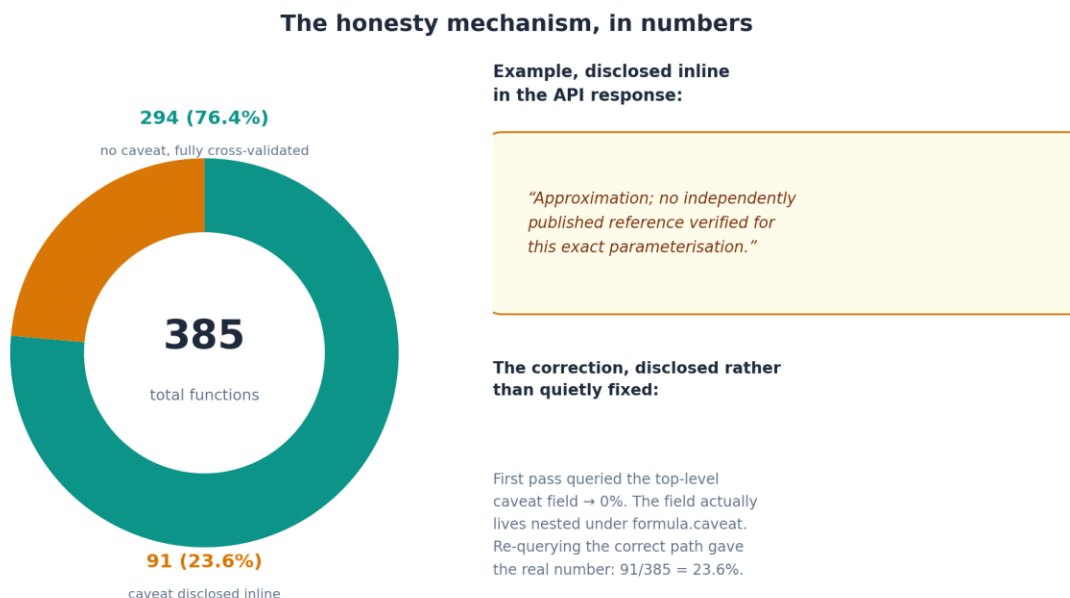
Bug 2 — a fix that broke something two files away. The CRR2 fix's own docstring quoted its new constant, `CRR2_INSTITUTION_ABSOLUTE_LIMIT_EUR`, inline. The portal's function-catalogue generator derives correct capitalisation for acronyms like "VaR" or "PD" by scanning docstrings for short all-caps words — and it doesn't know the difference between a genuine acronym and a fragment of a long constant name split on underscores. It saw "LIMIT" inside that constant, all-caps and six characters, and voted it in as the correct casing for the word "limit" — silently corrupting `crr2_large_exposure_limit`'s own display title from "CRR2 Large Exposure Limit" to "CRR2 Large Exposure LIMIT" on the public portal. The first attempted fix (strip anything inside double-backtick code spans before voting) overcorrected: it also stripped legitimate acronyms like KVA and MVA that happened to share a code span with an underscored variable in an unrelated formula, breaking *their* display titles in the process of fixing this one. That regression was caught before it was pushed — by regenerating the whole catalogue and diffing every `display_name` against the current live version, not just the one function being fixed — and the actual fix checks whether a candidate word is adjacent to an underscore in the source text, not whether it's inside a code span at all.

Neither of these was a follow-up someone requested. Both were found by treating "the code compiles and the happy-path test passes" as the start of verification, not the end of it — the same standard this whole article has been describing, applied to Claude Code's own recent work instead of to a formula from a textbook.

Four separately-reviewed, independently green pull requests came out of this pass. All four got folded into one `--no-ff` merge before landing on `master` — not a code change, an infrastructure-cost one: this repo's CDK pipeline runs a full synth-and-deploy cycle on every push to `master`, so merging four times would have triggered it four times for work that landed as one coherent unit. After the fold, every generated artifact (the portal catalogue, the MCP tool list, the SDK) was regenerated from scratch and diffed against what the merge had produced automatically — zero drift, confirming the fold hadn't silently dropped or garbled anything in the process of saving three pipeline runs.

Illustration 2 — the honesty mechanism, in numbers

pyvar doesn't claim all 385 functions are equally battle-tested. 91 of them carry a documented `caveat` field in the public function catalogue (`portal/functions.json`) — a modeling simplification or an independent-verification gap, disclosed to every API caller in the same response payload the function itself returns data in:



That 23.6% number is worth pausing on, because getting it required a real correction mid-session: the first pass at counting caveats checked a top-level `caveat` field on each function record and got 0% — which immediately looked wrong against what we already knew about the catalogue. The `caveat` field turns out to live nested under `formula.caveat`, not at the top level of the record. Re-querying the correct path gave $91/385 = 23.6\%$. We're disclosing that correction here rather than quietly fixing the number, because the whole point of this section is that a caveat rate is only meaningful if you trust how it was computed — and this article holds itself to the same standard it's describing.

The Jupyter integration: three bugs a naive build would have shipped

`pyvar-jupyter` — `%pyvar/%pyvar` IPython magics plus rich HTML display — is the newest of the three client surfaces, and its build is a good worked example of "verify by running it" as a discipline rather than a slogan.

Bug 1 — a silent method collision. The first draft defined `pyvar` as two separately-decorated methods on the same `Magics` subclass: one `@line_magic`, one `@cell_magic`. Python class bodies don't allow two methods with the same name to coexist — the second definition silently overwrites the first in the class namespace. Cell-magic support would have been dead code from the moment it was written, with no error anywhere to signal it. Fixed by using IPython's `@line_cell_magic` decorator on a single method that takes an optional `cell` parameter — the API IPython actually provides for exactly this case.

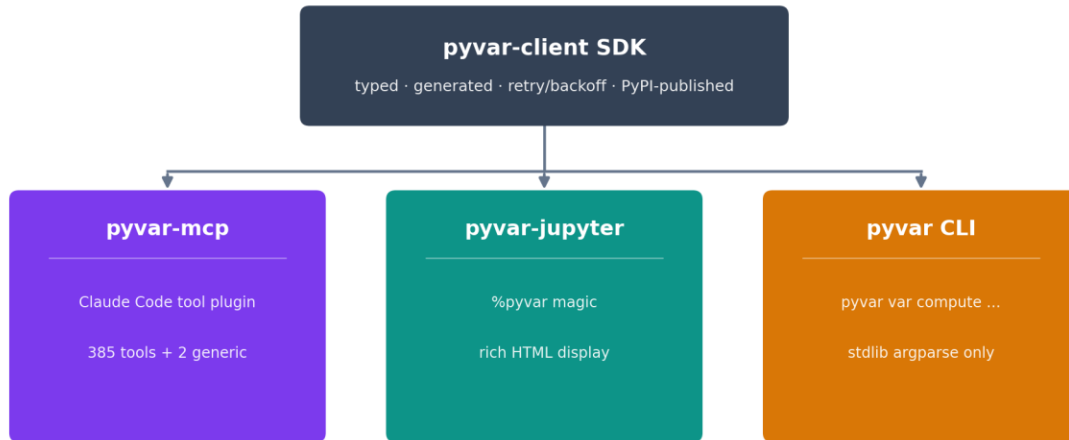
Bug 2 — a tokenizer that broke on its own inputs. IPython's `{expr}`-style variable expansion (`self.shell.var_expand`) interpolates a Python expression's `str()` representation directly into the magic line. For a list like `[0.01, -0.02]`, that representation contains a space after the comma. A naive `shlex.split()` or `whitespace.split()` on the expanded line chops that single value into two broken tokens. This wasn't caught by reasoning about it — it was caught by writing a regression test using a real `IPython.testing.globalipapp.get_ipython()` shell and watching it fail, which is what justified building a proper bracket/quote-depth-aware tokenizer (`tokenize_key_value_line`) instead of trusting a stdlib one-liner.

Bug 3 — a documented feature that doesn't exist. An early draft of the `02_basel_backtest.ipynb` example notebook stated that `%pyvar` cell magic supports `{{double-brace}}` interpolation of the cell body. It doesn't — `_invoke()` only ever expands the magic *line*, never a cell's body, by design (a cell body might contain code, not just parameter values, and expanding it blindly would be unsafe). The claim was caught not by review but by trying it: running the notebook's own assertion against the actual implementation failed immediately, which is exactly the situation "verified facts only" exists to catch before publication, not after.

While rebuilding that same notebook honestly, the synthetic backtest data — seeded to force exactly 3 breaches — actually produced 6 breaches once run through the real `traffic_light_backtesting` engine, because the random baseline itself contributes a few breaches by chance over 250 trading days. 3 forced breaches would have meant "green zone." The real, verified outcome is "yellow zone," and that's what the shipped notebook says — because a notebook that asserts an assumed outcome instead of the outcome you get from actually running the code isn't a very good advertisement for "verify by running it."

Illustration 3 — three integration surfaces, one SDK

Three integration surfaces, one SDK



None of the three duplicate HTTP or auth logic — a deliberate constraint, not an accident.

None of these three duplicate HTTP or auth logic. That's a deliberate constraint, not an accident: `pyvar-mcp`'s tool catalogue (`plugins/mcp/pyvar_mcp/_generated/functions.py`) and the plugin manifest (`.claude-plugin/marketplace.json`, `plugins/*`) are both generated directly from the repo's own source of truth (`.claude/skills/*`, `portal/functions.json`), with CI failing the build if committed output ever drifts from what regenerating produces. The same discipline that finds a formula bug — don't trust it, run it and check — applies to code generation too.

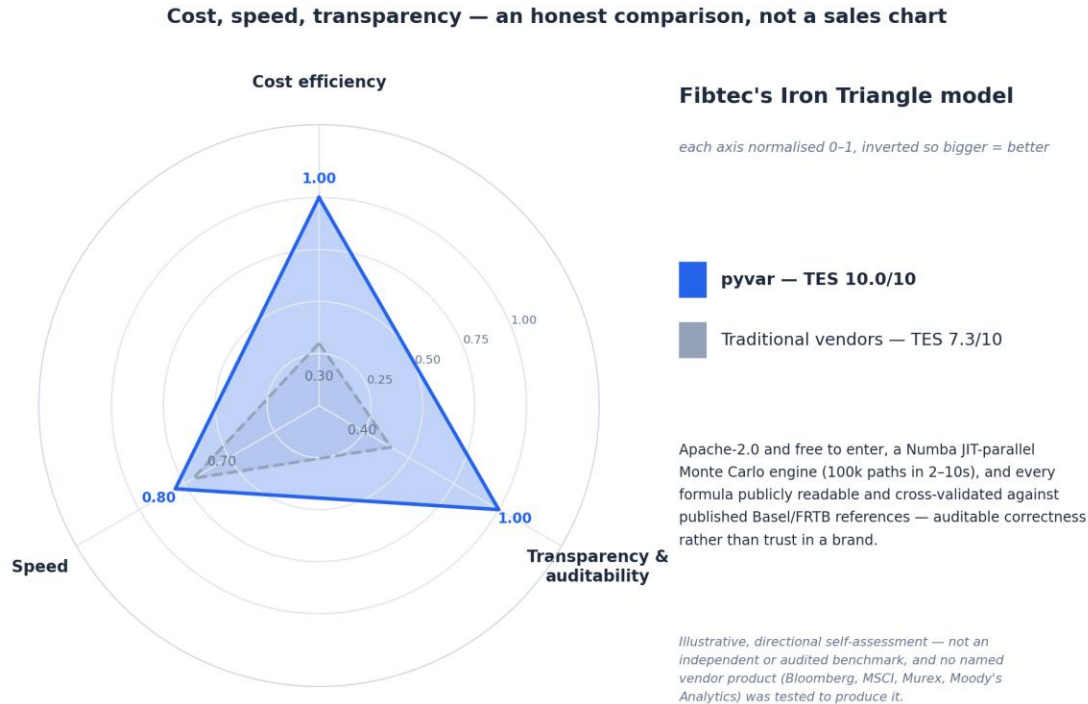
`pyvar-mcp` also ships 13 Claude Code skills (8 domain skills, one per risk domain, plus 5 covering pyvar's own architecture) — the difference between a model knowing a function *exists* and knowing *when* a risk team would actually reach for it.

What actually shipped, checkable today

- **385 risk functions, 8 domains** — traceable to the live OpenAPI schema and `portal/functions.json`.
 - **649 commits, 300 merged PRs**, of which **131 commits** carry a **Co-Authored-By: Claude** trailer — recounted directly against `git log` and the GitHub API immediately before publication, replacing a lower count from an earlier draft, for the same reason the 23.6% figure above got recounted mid-session: a number in an article about verifying numbers doesn't get a pass on being verified itself.
 - **`pyvar-client` on PyPI**, verified end-to-end — not just a green CI run, but a direct query against PyPI's own JSON API and a real `pip install` in a clean virtual environment.
 - **A public-launch security review**, not a one-time scan: org-wide 2FA, GitHub Secret Protection + Code Security (CodeQL) enabled and confirmed running, a full-history `gitleaks` scan across every commit on every branch, branch protection requiring PR + 6 named status checks, GitHub Actions pinned to commit SHA, least-privilege `permissions:` blocks on every workflow.
 - **`pyvar-mcp` submitted** to the `claude-plugins-community` marketplace — a real submission received by Anthropic's review team, decision pending as of publication.
-

Why it matters: cost, speed, and transparency

None of the rigor above means much commercially unless it adds up to something worth choosing over the alternative. pyvar.com scores itself against traditional enterprise risk vendors — Bloomberg, MSCI, Murex, Moody's Analytics and similar — using Fibtec's own Iron Triangle efficiency model: cost efficiency, speed, and transparency/auditability, each normalised 0–1 and inverted so a bigger shape reads as a better outcome.



To be direct about what this is and isn't: it's a self-scored, illustrative positioning, not an independent or audited benchmark, and no named vendor's product was actually tested to produce it. Traditional vendors bring decades of regulatory acceptance and institutional trust that an Apache-2.0 project launched this year hasn't earned yet — that's a real gap, and this chart doesn't dispute it.

What it does argue is narrower and, we think, solid: Apache-2.0 and free to enter, a Numba JIT-parallel Monte Carlo engine running 100k paths in 2–10 seconds, and every formula publicly readable and cross-validated against published Basel/FRTB references — auditable correctness in place of trust in a brand. That combination is the actual case for building this kind of software with an AI agent doing the heavy lifting: not that it's faster to type, but that verification thorough enough to publish becomes cheap enough to actually do.

The part we're not going to overclaim

This article, like the PRD it accompanies, isn't a case study about named enterprise customers running pyvar in production — there aren't any yet. It's a case study about what an AI coding agent can do end-to-end on a genuinely hard, regulatory-grade domain: find real Basel/Solvency II/EMIR/IRRBB bugs before launch, build and harden the AWS deployment pipeline around it, and then build its own way back into the ecosystem it came from via MCP.

The next real chapter — the one this article can't write yet — is whether someone outside Fibtec starts relying on `pyvar-mcp` inside their own Claude Code sessions. If that happens, it'll be checkable the same way everything above was: `git log`, PyPI, the live API, not a slide deck.

Sources

- **Live Platform:** <https://www.pyvar.com>
- **GitHub Repository:** <https://github.com/fibtecltd/pyvar>
- [CHANGELOG.md](#) (this repo) — regulatory fixes, PyPI publish history.
- [portal/functions.json](#) (this repo) — function/domain counts, caveat field structure.
- [pyvar-jupyter/](#) (this repo) — magics implementation, tests, example notebooks referenced above.
- [docs/prd-claude-partner-hub.md](#) (this repo) — the companion PRD this article was written alongside.
- `git log` (this repo) and the GitHub API — commit, PR, and Claude-co-authorship counts, verified at time of writing.
- [docs/caveat-triage-batch-plan.md](#) and [CHANGELOG.md](#) (this repo) — the 8-function caveat-triage follow-on pass (PRs #314–#318) described above.